

RELATÓRIO — PRÁTICA 1: DOCKER & DEPLOY PaaS

Disciplina: Sistemas Distribuídos — IFPA Campus Ananindeua

Equipe: Ian Lucas Lobato Barra da Silva e Leonardo Jacomini Barcos

Repositório: <https://github.com/percerval/sd-loadlab>

URL pública: <https://sd-loadlab.onrender.com>

Data da execução: 01/10/2026

1. Objetivo

Containerizar uma API REST em Python e publicá-la em uma plataforma PaaS, tornando-a acessível por uma URL pública.

2. Aplicação

API SD LoadLab desenvolvida em Flask com quatro endpoints: / (health check), /items (listagem com latência simulada de 20 ms), /items/<id> (consulta individual) e /cpu (processamento com hashes SHA-256). A rota de CPU aceita de 1 a 1.000.000 de iterações e rejeita entradas inválidas com HTTP 400. O Gunicorn usa 2 workers e 4 threads por worker. O servidor de desenvolvimento do Flask também suporta threads, mas não é recomendado para produção. Threads favorecem concorrência de I/O; processos permitem explorar múltiplos núcleos quando disponíveis.

3. Containerização (Dockerfile)

- **Imagem base python:3.12-slim:** imagem enxuta, reduz tamanho e tempo de build.
- **Cópia do requirements.txt antes do código:** aproveita o cache de camadas do Docker — as dependências só são reinstaladas se mudarem.
- **--no-cache-dir:** não guarda cache do pip, imagem menor.
- **Porta via variável PORT:** o PaaS define a porta em tempo de execução; o container se adapta.
- **.dockerignore:** evita copiar arquivos desnecessários para a imagem.

As Figuras 1 e 2 documentam o build concluído e a resposta HTTP do container local.

4. Deploy na plataforma PaaS (Render)

O código foi versionado e publicado no GitHub. No Render, o serviço foi construído pelo Dockerfile e disponibilizado com HTTPS. O log registrou **“Your service is live” em 01/10/2026 às 23:15:27 UTC**, com URL pública <https://sd-loadlab.onrender.com>.

O Gunicorn iniciou em 0.0.0.0:10000, com dois workers do tipo gthread. Embora o Render tenha definido WEB_CONCURRENCY=1, a opção explícita --workers 2 prevaleceu. O arquivo render.yaml especifica runtime Docker, plano Free e health check /.

Validação: o build Docker local e os nove testes automatizados passaram; pip check não encontrou conflitos de dependências. Na URL pública, /, /items, /items/1 e /cpu?n=20000 retornaram HTTP 200; /items/51 retornou 404 e /cpu?n=abc, 400. A listagem retornou os 50 produtos esperados.

A Figura 3 confirma o serviço Live, o plano Free e a URL pública no painel do Render.

5. Dificuldades e soluções

- **Validação da carga:** o código inicial convertia n sem tratar entradas inválidas e não limitava o trabalho solicitado. Foram adicionados tratamento de erro e limites, com resposta JSON e HTTP 400. Isso reduz o custo por requisição, mas não substitui rate limiting.
- **Publicação inicial:** a primeira tentativa de commit não tinha arquivos preparados; foi resolvida com git add. O primeiro push exigiu configurar o upstream com git push -u origin main.
- **Operação no PaaS:** a porta é obtida de PORT, o Gunicorn é iniciado com exec e os logs vão para stdout/stderr. Essas escolhas facilitam inicialização, encerramento e diagnóstico no ambiente hospedado.

6. Conclusão

Foi possível construir e executar a aplicação localmente com Docker e publicá-la no Render, com as rotas verificadas por HTTP. O container padroniza runtime e dependências, mas não torna hardware ou rede idênticos entre ambientes. O PaaS simplificou a publicação com HTTPS, enquanto Gunicorn e logs de execução forneceram a base operacional. A aplicação é um laboratório stateless, sem banco de dados real, adequado aos testes de carga da Prática 2.

Evidências de execução

```
percerval@debian:~/Projetos/sd-loadlab$ docker build -t sd-loadlab
[+] Building 18.4s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 423B
=> [internal] load metadata for docker.io/library/python:3.12-slim
=> [internal] load .dockerignore
=> => transferring context: 136B
=> [1/5] FROM docker.io/library/python:3.12-slim@sha256:eeb888e67610b37583880c7627e3931f087cba55a35810819e34a398f624a47
=> => resolve docker.io/library/python:3.12-slim@sha256:eeb888e67610b37583880c7627e3931f087cba55a35810819e34a398f624a47
=> sha256:5e62c0470a0973ad9561440c8a37984c3252eda1dd19e00ca7e76a37b62b5dc7 249B / 249B
=> sha256:e360d85fdc0f3910a4fe48ba2bb0865d5e618392ea8abfa424e66575e45273c0 12.12MB / 12.12MB
=> sha256:a510a9ea6d60786b7cefdbf9577945344c72879881de8d32ef07ec38111fb045 4.27MB / 4.27MB
=> => extracting sha256:a510a9ea6d60786b7cefdbf9577945344c72879881de8d32ef07ec38111fb045
=> => extracting sha256:e360d85fdc0f3910a4fe48ba2bb0865d5e618392ea8abfa424e66575e45273c0
=> => extracting sha256:5e62c0470a0973ad9561440c8a37984c3252eda1dd19e00ca7e76a37b62b5dc7
=> [internal] load build context
=> => transferring context: 63B
=> [2/5] WORKDIR /app
=> [3/5] COPY requirements.txt .
=> [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> [5/5] COPY app.py .
=> => exporting to image
=> => exporting layers
=> => exporting manifest sha256:18752b0157d3176a09fd4faef8cd1141174cca8531ea28c8f373c70b43545dae
=> => exporting config sha256:5240e942f6808957372bdf374f15b1fc053cb93aa63d577d72b2a691748961ed
=> => exporting attestation manifest sha256:38442f2185cdd3ce8df5a6d1850b6bf31b5c94a3f86f06145d8e495e53ffc444
=> => exporting manifest list sha256:a296d2d61206609f4ed8f251e3ec859c977da5721c97bf3efd6b55d473a36ac3
=> => naming to docker.io/library/sd-loadlab:latest
=> => unpacking to docker.io/library/sd-loadlab:latest
```

Figura 1 — Build local da imagem sd-loadlab, concluído com sucesso.

```
percerval@debian:~$ curl -i http://127.0.0.1:8000/
HTTP/1.1 200 OK
Server: gunicorn
Date: Thu, 01 Oct 2026 23:41:16 GMT
Connection: keep-alive
Content-Type: application/json
Content-Length: 62

{"host": "3418db373723", "status": "ok", "ts": 1798898076.9665883}
```

Figura 2 — Requisição a 127.0.0.1:8000, com HTTP 200, servidor Gunicorn e status ok.

My workspace

sd-loadlab

Deploys

Settings

Introducing Workflows

Changelog

Status

Collapse

My project

Production

sd-loadlab

Deploys

Search

+ New

Upgrade

?

WEB SERVICE

sd-loadlab

Docker

Free

Upgrade your instance

Connect

Manual Deploy

Service ID: srv-daveistsrm7s73bmrio0

percerval / sd-loadlab

main

85b9a74

Live

https://sd-loadlab.onrender.com

Your free instance will spin down with inactivity, which can delay requests by 50 seconds or more.

Upgrade now

Search deploys and commits

DEPLOY 1

TRIGGER

DURATION

feat: add SD LoadLab API with Docker and load tests

85b9a74

Deployed 20min ago

First Deploy

30, 0s

Figura 3 — Serviço sd-loadlab no Render: Docker, plano Free, status Live e URL pública.